



MODUL PEMROGRAMAN BERBASIS OBJEK 1

Pertemuan Ke-3

PHYTON BASE

Disusun Oleh :

Aziz Azindani, S.Pd., S,Kom., M.Kom

Modul Perkuliahan: Pertemuan 3

Polimorfisme dan Pewarisan

Tujuan Pembelajaran:

1. Memahami dan menerapkan konsep pewarisan dalam OOP.
 2. Memahami dan menerapkan konsep polimorfisme dalam OOP.
 3. Menggunakan pewarisan untuk memperluas fungsi kelas.
 4. Menggunakan polimorfisme untuk memanggil metode pada objek dengan cara yang berbeda.
-

I. Pengantar: Polimorfisme dan Pewarisan

Pada pertemuan ini, kita akan mempelajari dua konsep utama dalam OOP: **pewarisan** dan **polimorfisme**. Pewarisan memungkinkan kelas baru untuk mewarisi sifat dan metode dari kelas yang ada, sementara polimorfisme memungkinkan metode yang sama untuk digunakan pada objek dari kelas yang berbeda.

1. Pewarisan

Pewarisan adalah proses di mana satu kelas (kelas anak) mewarisi atribut dan metode dari kelas lain (kelas induk). Ini memungkinkan kita untuk menggunakan kembali kode yang ada dan memperluas fungsionalitasnya.

A. Konsep Dasar Pewarisan

- **Kelas Induk (Super Class):** Kelas yang menyediakan atribut dan metode yang akan diwarisi.
- **Kelas Anak (Sub Class):** Kelas yang mewarisi atribut dan metode dari kelas induk.

B. Mengapa Menggunakan Pewarisan?

- **Reuse:** Memungkinkan penggunaan kembali kode yang ada.
- **Maintenance:** Mempermudah pemeliharaan dengan mengurangi duplikasi kode.
- **Extensibility:** Mempermudah penambahan fitur baru.

C. Contoh Pewarisan dalam Kehidupan Nyata

Pikirkan hierarki kendaraan: `Kendaraan` adalah kelas induk, sementara `Mobil` dan `SepedaMotor` adalah kelas anak yang mewarisi sifat dasar dari `Kendaraan`.

Contoh Implementasi:

```
class Kendaraan:
    def __init__(self, nama, tahun):
        self.nama = nama
        self.tahun = tahun

    def info(self):
        return f"Kendaraan {self.nama}, tahun {self.tahun}"

class Mobil(Kendaraan):
    def __init__(self, nama, tahun, tipe):
        super().__init__(nama, tahun)
        self.tipe = tipe

    def info(self):
        return f"Mobil {self.nama}, tahun {self.tahun}, tipe {self.tipe}"

# Praktik
mobil1 = Mobil("Toyota", 2022, "SUV")
print(mobil1.info())
```

2. Polimorfisme

Polimorfisme memungkinkan objek dari kelas yang berbeda untuk diperlakukan sebagai objek dari kelas induk mereka. Ini membuat kode lebih fleksibel dan dapat beradaptasi dengan berbagai tipe objek.

A. Konsep Dasar Polimorfisme

- **Polimorfisme Ad Hoc (Function Overloading):** Metode dengan nama yang sama melakukan tugas yang berbeda.
- **Polimorfisme Parametrik (Function Overriding):** Metode yang diwarisi dari kelas induk diubah dalam kelas anak.

B. Mengapa Menggunakan Polimorfisme?

- **Flexibility:** Memungkinkan penggunaan antarmuka yang konsisten untuk berbagai tipe objek.
- **Interchangeability:** Memudahkan penggantian objek dalam sistem tanpa mengubah kode yang menggunakan objek tersebut.

C. Contoh Polimorfisme dalam Kehidupan Nyata

Bayangkan sebuah sistem pembayaran di mana metode `bayar` dapat digunakan untuk berbagai jenis pembayaran seperti `KartuKredit`, `TransferBank`, dan `E-Wallet`. Setiap metode pembayaran dapat memiliki implementasi `bayar` yang berbeda, tetapi dipanggil dengan cara yang sama.

Contoh Implementasi:

```
class Pembayaran:
    def bayar(self, jumlah):
        pass

class KartuKredit(Pembayaran):
    def bayar(self, jumlah):
        print(f"Membayar {jumlah} dengan Kartu Kredit")

class TransferBank(Pembayaran):
    def bayar(self, jumlah):
        print(f"Membayar {jumlah} dengan Transfer Bank")

# Praktik
pembayaran1 = KartuKredit()
pembayaran1.bayar(5000)

pembayaran2 = TransferBank()
pembayaran2.bayar(5000)
```

II. Pewarisan: Teori dan Praktik Terintegrasi

A. Definisi dan Penerapan Pewarisan

Pewarisan memungkinkan kelas baru untuk mengambil sifat dan metode dari kelas yang ada. Kelas anak dapat menambah atau mengubah fungsionalitas kelas induk.

1. Membuat dan Menggunakan Kelas Anak

- **Kelas Induk:** Definisikan atribut dan metode yang ingin diwarisi.
- **Kelas Anak:** Warisi kelas induk dengan sintaks `class Anak(KelasInduk)` dan tambahkan atau ubah fungsionalitas.

Contoh Implementasi:

```
class Hewan:
    def __init__(self, nama):
        self.nama = nama

    def suara(self):
        return "Hewan mengeluarkan suara"

class Anjing(Hewan):
    def suara(self):
        return "Anjing menggonggong"

class Kucing(Hewan):
    def suara(self):
        return "Kucing mengeong"

# Praktik
anjing = Anjing("Buddy")
kucing = Kucing("Whiskers")
print(anjing.suara())
print(kucing.suara())
```

2. Menggunakan `super()` untuk Mengakses Kelas Induk

Metode `super()` digunakan untuk memanggil metode dari kelas induk dari dalam kelas anak.

Contoh Implementasi:

```
class Kendaraan:
    def __init__(self, nama, tahun):
        self.nama = nama
        self.tahun = tahun

    def info(self):
        return f"Kendaraan {self.nama}, tahun {self.tahun}"

class Motor(Kendaraan):
    def __init__(self, nama, tahun, jenis):
        super().__init__(nama, tahun)
        self.jenis = jenis

    def info(self):
        return f"Motor {self.nama}, tahun {self.tahun}, jenis {self.jenis}"

# Praktik
motor1 = Motor("Yamaha", 2021, "Sport")
print(motor1.info())
```

III. Polimorfisme: Teori dan Praktik Terintegrasi

A. Definisi dan Penerapan Polimorfisme

Polimorfisme memungkinkan satu antarmuka untuk diterapkan oleh beberapa kelas yang berbeda. Ini bisa dilakukan melalui method overriding.

1. Overriding Metode dari Kelas Induk

Kelas anak dapat menggantikan metode dari kelas induk untuk memberikan implementasi yang berbeda.

Contoh Implementasi:

```
class Pembayaran:
    def bayar(self, jumlah):
        print("Membayar sejumlah", jumlah)

class EWallet(Pembayaran):
    def bayar(self, jumlah):
        print(f"Membayar {jumlah} dengan E-Wallet")

class KartuDebit(Pembayaran):
    def bayar(self, jumlah):
        print(f"Membayar {jumlah} dengan Kartu Debit")

# Praktik
pembayaran1 = EWallet()
```

```
pembayaran2 = KartuDebit()
pembayaran1.bayar(5000)
pembayaran2.bayar(5000)
```

2. Menggunakan Polimorfisme untuk Interchangeability

Dengan polimorfisme, kita dapat membuat daftar objek dari kelas yang berbeda dan memanggil metode yang sama pada setiap objek.

Contoh Implementasi:

```
class Hewan:
    def suara(self):
        pass

class Anjing(Hewan):
    def suara(self):
        print("Anjing menggonggong")

class Kucing(Hewan):
    def suara(self):
        print("Kucing mengeong")

# Praktik
hewan1 = Anjing()
hewan2 = Kucing()

daftar_hewan = [hewan1, hewan2]
for hewan in daftar_hewan:
    hewan.suara()
```

IV. Latihan dan Praktik Mandiri

A. Praktik Mandiri: Pewarisan dengan Kelas `Elektronik`

Tugas:

1. Buat kelas `Elektronik` dengan atribut `nama` dan `merek`.
2. Buat kelas `Televisi` dan `Kulkas` yang mewarisi `Elektronik`.
3. Tambahkan metode `info` di setiap kelas untuk menampilkan informasi.

Contoh Solusi:

```
class Elektronik:
    def __init__(self, nama, merek):
        self.nama = nama
        self.merek = merek

    def info(self):
        return f"{self.nama} oleh {self.merek}"

class Televisi(Elektronik):
    def __init__(self, nama, merek, ukuran):
        super().__init__(nama, merek)
```

```

        self.ukuran = ukuran

    def info(self):
        return f"Televisi {self.nama} oleh {self.merek}, ukuran {self.ukuran} inci"

class Kulkas(Elektronik):
    def __init__(self, nama, merek, kapasitas):
        super().__init__(nama, merek)
        self.kapasitas = kapasitas

    def info(self):
        return f"Kulkas {self.nama} oleh {self.merek}, kapasitas {self.kapasitas} liter"

# Praktik
tv = Televisi("Model X", "Samsung", 55)
kulkas = Kulkas("Model Y", "LG", 300)
print(tv.info())
print(kulkas.info())

```

B. Praktik Mandiri: Polimorfisme dengan Kelas Kendaraan

Tugas:

1. Buat kelas abstrak `Kendaraan` dengan metode abstrak `bergerak`.
2. Buat kelas `Mobil` dan `SepedaMotor` yang mengimplementasikan `Kendaraan`.
3. Gunakan polimorfisme untuk memanggil metode `bergerak` pada objek `Mobil` dan `SepedaMotor`.

Contoh Solusi:

```

from abc import ABC, abstractmethod

class Kendaraan(ABC):
    @abstractmethod
    def bergerak(self):
        pass

class Mobil(Kendaraan):
    def bergerak(self):
        print("Mobil bergerak di jalan")

class SepedaMotor(Kendaraan):
    def bergerak(self):
        print("Sepeda motor bergerak di jalan")

# Praktik
mobil = Mobil()
sepeda_motor = SepedaMotor()

daftar_kendaraan = [mobil, sepeda_motor]
for kendaraan in daftar_kendaraan:
    kendaraan.bergerak()

```

V. Rangkuman

1. **Pewarisan** memungkinkan penggunaan kembali dan perluasan kode dengan mewarisi atribut dan metode dari kelas induk.
 2. **Polimorfisme** memungkinkan metode yang sama untuk bekerja pada objek dari kelas yang berbeda, membuat kode lebih fleksibel.
 3. **Method overriding** memungkinkan kelas anak untuk menyediakan implementasi spesifik dari metode kelas induk.
 4. **Polimorfisme dan pewarisan** bekerja bersama untuk mendukung prinsip OOP dalam pengembangan perangkat lunak.
-

VI. Pertanyaan dan Diskusi

1. Diskusikan bagaimana pewarisan dapat mempermudah pengembangan perangkat lunak.
 2. Berikan contoh nyata di mana polimorfisme membantu dalam desain sistem.
 3. Apa manfaat utama menggunakan `super()` dalam kelas anak?
-

VII. Lampiran Kode

Kode Pewarisan:

```
class Kendaraan:
    def __init__(self, nama, tahun):
        self.nama = nama
        self.tahun = tahun

    def info(self):
        return f"Kendaraan {self.nama}, tahun {self.tahun}"

class Mobil(Kendaraan):
    def __init__(self, nama, tahun, tipe):
        super().__init__(nama, tahun)
        self.tipe = tipe

    def info(self):
        return f"Mobil {self.nama}, tahun {self.tahun}, tipe {self.tipe}"

# Praktik
mobil1 = Mobil("Toyota", 2022, "SUV")
print(mobil1.info())
```

Kode Polimorfisme:

```
class Pembayaran:
    def bayar(self, jumlah):
        print("Membayar sejumlah", jumlah)

class EWallet(Pembayaran):
```



```

        def bayar(self, jumlah):
            print(f"Membayar {jumlah} dengan E-Wallet")

class KartuDebit(Pembayaran):
    def bayar(self, jumlah):
        print(f"Membayar {jumlah} dengan Kartu Debit")

# Praktik
pembayaran1 = EWallet()
pembayaran2 = KartuDebit()
pembayaran1.bayar(5000)
pembayaran2.bayar(5000)

```

Kode Latihan:

```

class Elektronik:
    def __init__(self, nama, merek):
        self.nama = nama
        self.merek = merek

    def info(self):
        return f"{self.nama} oleh {self.merek}"

class Televisi(Elektronik):
    def __init__(self, nama, merek, ukuran):
        super().__init__(nama, merek)
        self.ukuran = ukuran

    def info(self):
        return f"Televisi {self.nama} oleh {self.merek}, ukuran {self.ukuran} inci"

class Kulkas(Elektronik):
    def __init__(self, nama, merek, kapasitas):
        super().__init__(nama, merek)
        self.kapasitas = kapasitas

    def info(self):
        return f"Kulkas {self.nama} oleh {self.merek}, kapasitas {self.kapasitas} liter"

# Praktik
tv = Televisi("Model X", "Samsung", 55)
kulkas = Kulkas("Model Y", "LG", 300)
print(tv.info())
print(kulkas.info())

```

```

from abc import ABC, abstractmethod

class Kendaraan(ABC):
    @abstractmethod
    def bergerak(self):
        pass

class Mobil(Kendaraan):
    def bergerak(self):
        print("Mobil bergerak di jalan")

class SepedaMotor(Kendaraan):

```

```
        def bergerak(self):
            print("Sepeda motor bergerak di jalan")

# Praktik
mobil = Mobil()
sepeda_motor = SepedaMotor()

daftar_kendaraan = [mobil, sepeda_motor]
for kendaraan in daftar_kendaraan:
    kendaraan.bergerak()
```